

11376/8890 Computer Vision and Image Analysis - Computer Laboratory

Week 5 - Image Matching with Local Invariant Features

[15/Mar/2026 Dr Min Wang]

Learning Objectives

In this lab, you will learn how to perform image matching using local invariant image features. By the end of the lab, you should be able to:

- Detect interest points using the Harris Corner Detector
- Extract local feature descriptors using SIFT
- Match features between two images
- Visualise the correspondence between images

This is a fundamental technique used in many computer vision applications such as:

- Image stitching (panoramas)
- Object recognition
- Image registration
- Visual localisation

Feature-Based Image Matching

Instead of comparing images pixel by pixel, modern computer vision systems match images using distinctive local features. The typical pipeline is:

- Step 1: Feature Detection. This is to find keypoints (distinctive locations) in an image, for example, the Corners, Blobs, or Edges. Good keypoints should be repeatable, distinctive, and robust to noise.
- Step 2: Feature Description. For each detected keypoint, we compute a descriptor that describes the local image region. A descriptor is usually a vector representation of the image patch around the keypoint. For example, the SIFT descriptor which is a 128-dimensional vector.
- Step 3: Feature Matching. Descriptors from two images are compared using a distance metric. Common metrics include Euclidean distance (L2) and Cosine similarity. The closest descriptors are considered matches.
- Step 4: Match Filtering. Some matches are incorrect due to noise or similar textures. Techniques such as Lowe's ratio test are used to remove false matches.

Software Requirements

We will use:

- Python
- OpenCV
- NumPy
- Matplotlib

Install the required libraries:

```
In [31]: # pip install opencv-python matplotlib numpy
```

Code Example:

1 – Load and Visualize Images

Now let's load two images and convert them to grayscale. Most feature detection algorithms operate on grayscale images, because they analyse intensity changes rather than color information. Grayscale images simplify computation while preserving important structure information.

```
In [23]: import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load two images
img1 = cv2.imread('canberra1.png')
img2 = cv2.imread('canberra2.png')

# Convert to grayscale
gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

# Display the first image
plt.imshow(gray1, cmap='gray')
plt.title("Image 1 (Grayscale)")
plt.axis("off")
plt.show()

plt.imshow(gray2, cmap='gray')
plt.title("Image 2 (Grayscale)")
plt.axis("off")
plt.show()
```

Image 1 (Grayscale)



Image 2 (Grayscale)



2 - Detect Harris Corners

Now we need to detect corner points in the image. Corners are useful features because:

- They are distinctive
- They are stable under small transformations
- They provide reliable points for matching

The Harris Corner Detector identifies locations where intensity changes in two directions. Mathematically, the Harris response is:

$$R = \det(M) - k(\text{trace}(M))^2$$

where M is the auto-correlation matrix and k is a constant (typically 0.04). Large positive values of R indicate corners.

```
In [24]: # Harris corner detection requires float32 images
gray1_float = np.float32(gray1)

# Apply Harris Corner Detector
dst = cv2.cornerHarris(gray1_float, blockSize=2, ksize=3, k=0.04)

# Create a copy for visualization
img_harris = img1.copy()

# Mark strong corners in red
img_harris[dst > 0.05 * dst.max()] = [0, 0, 255]

# Display results
plt.imshow(cv2.cvtColor(img_harris, cv2.COLOR_BGR2RGB))
plt.title("Detected Harris Corners")
plt.axis("off")
plt.show()
```

Detected Harris Corners



Key parameters:

- `blockSize`: neighborhood size used for corner detection
- `ksize`: aperture size for Sobel operator
- `k`: Harris detector constant

The threshold $0.05 * \text{dst.max}()$ selects strong corner responses.

3 – Extract SIFT Features

Now we detect keypoints and compute SIFT descriptors. Unlike Harris corners, SIFT features are scale and rotation invariant.

SIFT works by:

- Detecting scale-space extrema
- Selecting stable keypoints
- Assigning dominant orientations
- Computing a 128-dimensional descriptor

```
In [25]: # Create SIFT detector
sift = cv2.SIFT_create()

# Detect keypoints and compute descriptors
kp1, des1 = sift.detectAndCompute(gray1, None)
kp2, des2 = sift.detectAndCompute(gray2, None)

print("Number of keypoints in image 1:", len(kp1))
print("Number of keypoints in image 2:", len(kp2))
```

Number of keypoints in image 1: 3238

Number of keypoints in image 2: 4311

Here:

- kp1 contains keypoint objects
- des1 contains feature descriptors

Each descriptor is a 128-dimensional vector representing gradient information around the keypoint.

4 – Visualize SIFT Keypoints

Now, let's display detected SIFT keypoints on the image. This helps us understand where the algorithm identifies distinctive features.

```
In [26]: img_sift = cv2.drawKeypoints(
    img1,
    kp1,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

plt.imshow(cv2.cvtColor(img_sift, cv2.COLOR_BGR2RGB))
plt.title("SIFT Keypoints")
plt.axis("off")
plt.show()
```

SIFT Keypoints



Here, each keypoint visualization includes:

- Position
- Scale (circle size)
- Orientation

These properties allow SIFT to be invariant to rotation and scale changes.

5 – Match Feature Descriptors

Find corresponding keypoints between two images. We compare descriptors using Euclidean distance. The Brute Force Matcher compares every descriptor from one image with every descriptor from the other image.

```
In [27]: bf = cv2.BFMatcher(cv2.NORM_L2, crossCheck=True)

matches = bf.match(des1, des2)

# Sort matches based on descriptor distance
matches = sorted(matches, key=lambda x: x.distance)

print("Total matches found:", len(matches))
```

Total matches found: 1392

Explanation:

- cv2.NORM_L2 computes Euclidean distance
- crossCheck=True ensures matches are mutual

Lower distance means better match

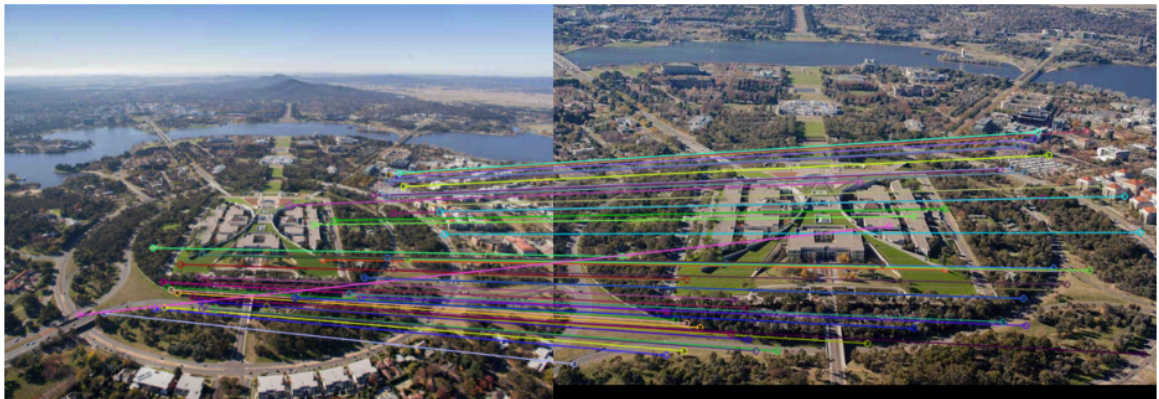
6 – Visualize Feature Matches

Now, let's visualise correspondences between two images. Each line represents a matching pair of keypoints.

```
In [28]: matched_img = cv2.drawMatches(
    img1, kp1,
    img2, kp2,
    matches[:50], None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

plt.figure(figsize=(12,6))
plt.imshow(cv2.cvtColor(matched_img, cv2.COLOR_BGR2RGB))
plt.title("Top 50 Feature Matches")
plt.axis("off")
plt.show()
```

Top 50 Feature Matches



Here, only the best 50 matches are shown to keep the visualisation clear. Correct matches indicate that the algorithm successfully identified corresponding locations in both images.

7 – Improve Matching Using the Ratio Test

Now we want to improve matching by removing incorrect matches.

The Lowe Ratio Test compares the best match with the second-best match. If the best match is much better, it is considered reliable.

```
In [29]: bf = cv2.BFMatcher()

matches = bf.knnMatch(des1, des2, k=2)

good_matches = []

for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good_matches.append(m)

print("Good matches after ratio test:", len(good_matches))
```

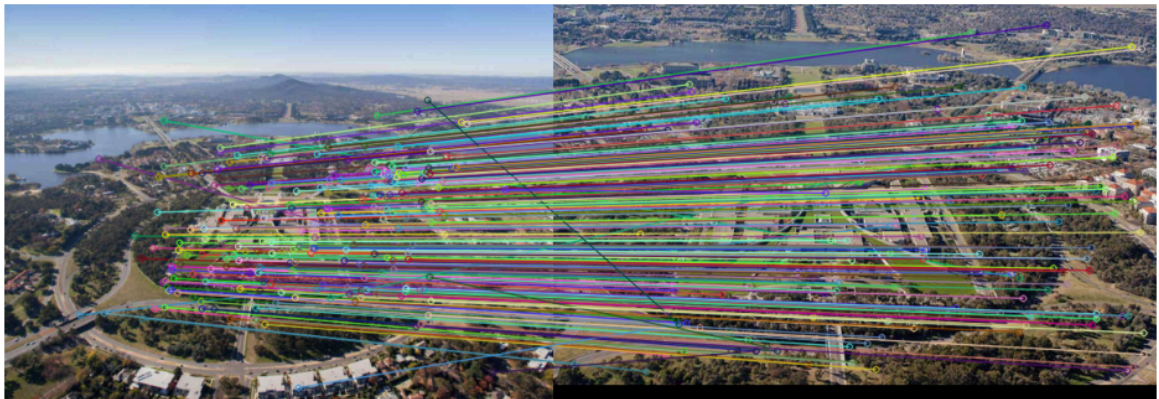
Good matches after ratio test: 237

Here, the condition: $m.distance < 0.75 * n.distance$, ensures that the best match is significantly better than the second-best match, reducing ambiguous matches.

```
In [30]: # Visualize Filtered Matches
matched_img = cv2.drawMatches(
    img1, kp1,
    img2, kp2,
    good_matches, None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

plt.figure(figsize=(12,6))
plt.imshow(cv2.cvtColor(matched_img, cv2.COLOR_BGR2RGB))
plt.title("Filtered Matches (Ratio Test)")
plt.axis("off")
plt.show()
```

Filtered Matches (Ratio Test)



Discussion

1. Why are corners good features for image matching?
2. Why does SIFT perform better than Harris corners for matching?
3. What is the purpose of the ratio test?
4. What types of transformations is SIFT invariant to?
5. Why is Euclidean distance used to compare SIFT descriptors?

In []: